

Week 4

```
GNU nano 8.7.1                                     week-4.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2;

    pid1 = fork();
    if (pid1 == 0) {
        printf("Child1 PID: %d\n", getpid());
        sleep(2);
        exit(0);
    }

    pid2 = fork();
    if (pid2 == 0) {
        printf("Child2 PID: %d\n", getpid());
        sleep(4);
        exit(0);
    }

    // Parent process
    printf("Parent PID: %d\n", getpid());

    // Using wait() - waits for any child
    int status;
    wait(&status);
    printf("One child finished using wait()\n");

    // Using waitpid() - waits for specific child
    waitpid(pid2, &status, 0);
    printf("Second child finished using waitpid()\n");

    // Zombie scenario: comment out wait/waitpid above
    // Then check with `ps -l` to see <defunct> process
    // To fix zombie: always use wait/waitpid
    return 0;
}
```

```
adith@KesavaSatish:~$ gcc week-4.c
```

```
adith@KesavaSatish:~$ ./a.out
```

```
Parent PID: 673
```

```
Child1 PID: 674
```

```
Child2 PID: 675
```

```
One child finished using wait()
```

```
Second child finished using waitpid()
```